

9-12

encryption

- la cifratura a livello applicativo funziona ma lascia scoperti tutti i layer sottostanti, come trasporto, internet, e link.
- la cifratura a livello di trasporto è possibile cifrare il protocollo di TCP, ma internet si appoggia a middleboxes, host che devono poter guardare dentro all'header TCP, firewalls e traffic shapers non potrebbero funzionare
- se applico una cifratura a livello di connessione dovrei cambiare la modalità di connessione a livello di tutti i router, il problema di questa modalità è la scalabilità.
- per applicare cifratura a livello IP è possibile con una VPN, a un pacchetto IP viene incapsulato all'interno di un altro pacchetto

SSL(secure socket layer)/TLS(transport layer security): protocollo sopra a TCP per renderlo sicuro, ne hanno fatto molte versioni.

protocolli 'S': molti protocolli sono stati rimpiazzati con una loro versione sicura: HTTPS, POP3S, IMAPS che funzionano su porte diverse.

SSL/TLS

fornisce i seguenti servizi:

1. autenticazione attraverso certificati X.509
2. segretezza attraverso scambio di chiavi con chiave simmetrica
3. integrità con HMAC

TLS 1.2

protocollo di encapsulation del record

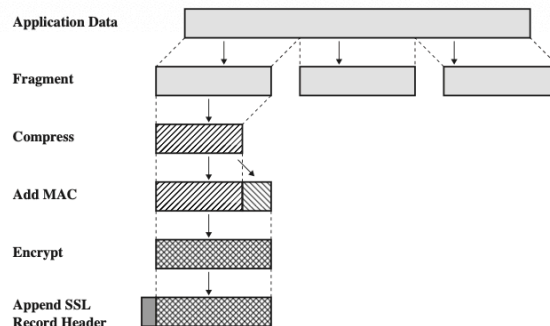


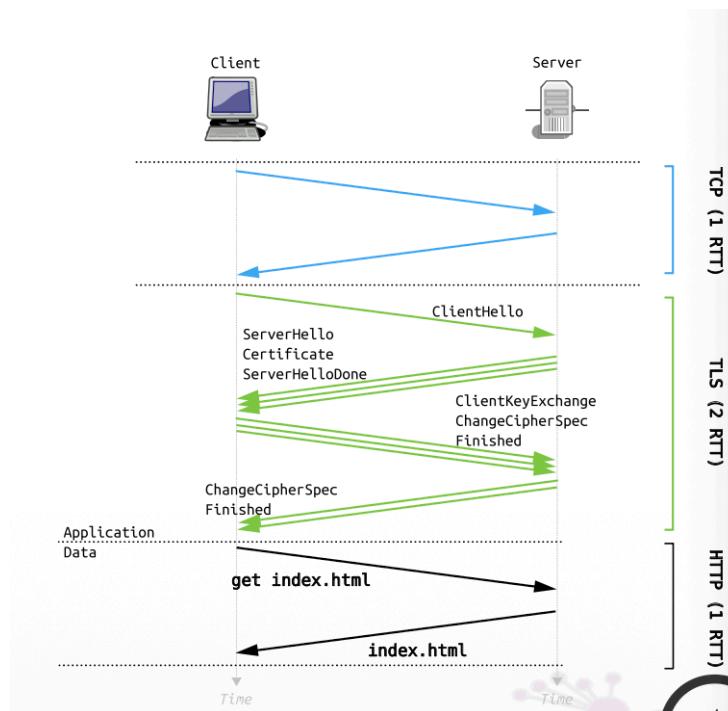
Figure 17.3 SSL Record Protocol Operation

fornisce 2 proprietà:

- **la connessione è privata:** è usata crittografia a chiave simmetrica, le chiavi per la crittografia simmetrica sono generate unicamente per ogni connessione e sono negoziate con un altro protocollo sicuro.
- **la connessione è affidabile:** il trasporto del messaggio include cechk dell'integrità del messaggio con MAC, con funzioni di hashing sicure come SHA-256

TSL permette la negoziazione di chiavi e funzioni di hashing prima di trasmettere i dati veri e propri. E' critico in quanto è possibile un attacco MiTM.

TLS handshake protocol

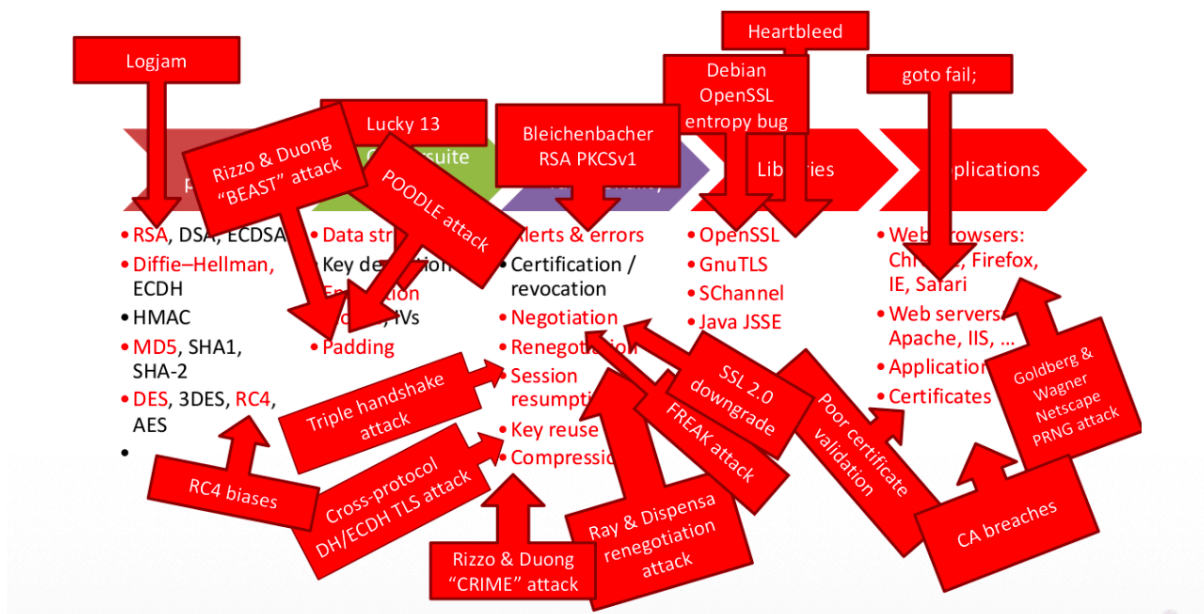


- la risposta del clientHello conta come ACK del TCP del client
- dentro HTTPS certificate c'è tutta la catena di certificati, fino a prima della root, che dovrebbe già essere nel browser.
- changecypherspec conferma la conclusione dello scambio delle chiavi, dentro finished c'è l'hash dei messaggi precedenti per confermare che non siano stati modificati da un MiTM

one way outhentication: spesso solamente il server invia un certificato valido per autenticarsi al browser, quindi il client è sicuro dell'identità del server ma non il contrario. Si può comunque negoziare una chiave simmetrica per rendere la connessione sicura in entrambe le direzioni → mutua autenticazione



la sicurezza di TLS dipende da molti componenti, la superficie di attacco è molto ampia, di seguito le vulnerabilità trovate.



TLS 1.3 modifiche

- solo 5 algoritmi di crittografia/autenticazione con un solo block scheme
- handshake iniziale composto da un solo RTT
- la sessione può ricominciare con 0 RTT
- forward secrecy è obbligatoria: impedire di fare offline brute force. per fare ciò viene negoziata una chiave pubblica diversa per ogni sessione generalmente con Diffie-Hellman.

$B \rightarrow A : CERT_B$

$A : \text{verify } CERT_B$

$A \rightarrow B : m, g, n(\text{ephemeralDH})$

$B \rightarrow A : \{m, g, n, r\}_{K_B^{-1}}$

$A, B : K = m^b = r^a$

$A, B = AES/HMAC \text{ con } K$

$A, B : \text{cancellano } a, b$

$A \rightarrow B : \text{password based authentication.}$



se l'attaccante entrasse in possesso della chiave privata del server, non è stata usata per generare K , è stata usata una chiave effimera

per verificare la password spesso è usato un protocollo di challenge-response, che può essere soggetto a offline-brute force: è quindi affidata a un protocollo del layer superiore.

- HTTPS manda la password all'interno di TLS senza challenge
- 802.11 usa MSCHAPv2 che usa un protocollo di challenge/response.

network sockets

quando un'applicazione ha bisogno di mandare o ricevere dati dalla rete usa una un'API che crea, usa e organizza i sockets.

Gestione dei Dati e Indirizzi

- **Byte Order:** Esiste una differenza tra come i computer memorizzano i dati (Little-endian vs Big-endian). I protocolli di rete (Internet) usano il **Big-endian**.
 - Funzioni di conversione necessarie: `htons()`, `htonl()` (Host to Network) e `ntohs()`, `ntohl()` (Network to Host).
- **Strutture Indirizzo:**
 - `struct sockaddr_in`: Struttura specifica per indirizzi IPv4 (include famiglia, porta, indirizzo IP).
 - `struct sockaddr`: Struttura generica usata per passare indirizzi alle funzioni socket (richiede casting).

API Socket TCP (Funzioni Chiave)

Il flusso tipico prevede:

1. **socket():** Crea il socket (specificando IPv4/IPv6 e TCP/UDP).
2. **bind():** (Lato Server) Associa il socket a un IP e una porta specifici.
3. **listen():** (Lato Server) Converte il socket in passivo e imposta la coda di connessioni.
4. **connect():** (Lato Client) Avvia la connessione (three-way handshake) verso il server.
5. **accept():** (Lato Server) Accetta una richiesta in coda e crea un *nuovo* socket dedicato a quel client specifico.
6. **send() / recv():** Invio e ricezione dati (simili a read/write ma con opzioni specifiche).
7. **close():** Termina la connessione.

API Socket UDP (Funzioni Chiave)

Essendo connection-less, il flusso è più semplice:

1. **socket():** Crea il socket.
2. **bind():** (Lato Server) Necessario per ricevere su una porta specifica.
3. **sendto():** Invia dati specificando ogni volta l'indirizzo del destinatario.
4. **recvfrom():** Riceve dati e restituisce anche l'indirizzo del mittente (per poter rispondere).

Server Concorrenti (Concurrent Servers)

- **Iterative Server:** Gestisce un client alla volta. Gli altri devono aspettare.
- **Concurrent Server:** Può gestire più client contemporaneamente.
 - **Metodo `fork()`:** Quando `accept()` ritorna una nuova connessione, il server padre chiama `fork()` per creare un processo figlio.
 - Il **figlio** gestisce la comunicazione con il client e poi termina.

- Il **padre** chiude il socket di connessione (affidato al figlio) e torna subito in ascolto (`accept`) per nuovi client.